

1-Task Title: Coin Change (Greedy Algorithm method)

Code:

```
coin=int(input("Amount of coin change-> "))
A=[25,10,5,1]
a=0
B=[]
count=0
while coin!=0:
    if A[a]<=coin:
        count+=1
        coin-=A[a]
        if coin==0:
            B.append((A[a],count))
    else:
        B.append((A[a],count))
        a+=1
        count=0
print(B)
```

Output:

```
Amount of coin change-> 37
[(25, 1), (10, 1), (5, 0), (1, 2)]
PS F:\RUET CSE LAB\Discrete Mathematics 2102\6. 17-02-2024>
```

Discussion:

In this program, I have tried to solve the coin change problem in a greedy approach. A greedy Algorithm can give a better solution to a problem, but it does not always give the optimal solution. Here, I must show the total number of coins needed for each coin in the list.

2-Task Title: Time Comparison between Bubble & Quick Sort Algorithm

Code:

<pre>class Sorting_Algorithm: def __init__(self): pass</pre>	
<pre>def Quick_Sort(self,A): if len(A)<=1: return A pivot=int(len(A)/2-1) b=[] c=[] for i in range(len(A)): if i==pivot: continue if A[i]<=A[pivot]: b.append(A[i]) else: c.append(A[i]) Sorted_B=self.Quick_Sort(b) Sorted_C=self.Quick_Sort(c) Sorted_A=[] for i in range(len(Sorted_B)): Sorted_A.append(Sorted_B[i]) Sorted_A.append(A[pivot]) for i in range(len(Sorted_C)): Sorted_A.append(Sorted_C[i]) return Sorted_A</pre>	<pre>def Bubble_Sort(self,c): for j in range(0,len(c)): for k in range(0,len(c)-1): if c[k]>c[k+1]: temp=c[k] c[k]=c[k+1] c[k+1]=temp return c</pre>

```
from datetime import datetime
from Sorting_Algorithm import Sorting_Algorithm

sort=Sorting_Algorithm()
with open("Input.txt",mode="r") as file1:
    num=file1.readlines()
with open("Output.txt",mode="a") as file:
    file.write("FOR Quicksort    FOR BUBBLE SORT\n")
    B=[]
    for r in num:
        if r!="\n":
            B.append(int(r.strip("\n")))
        else:
            A=B
            start_time=datetime.now()
            x=sort.Quick_Sort(A)
            finish_time=datetime.now()
            k=str(finish_time-start_time)
            file.write(k+'    ')
            start_time=datetime.now()
            y=sort.Bubble_Sort(A)
```

```

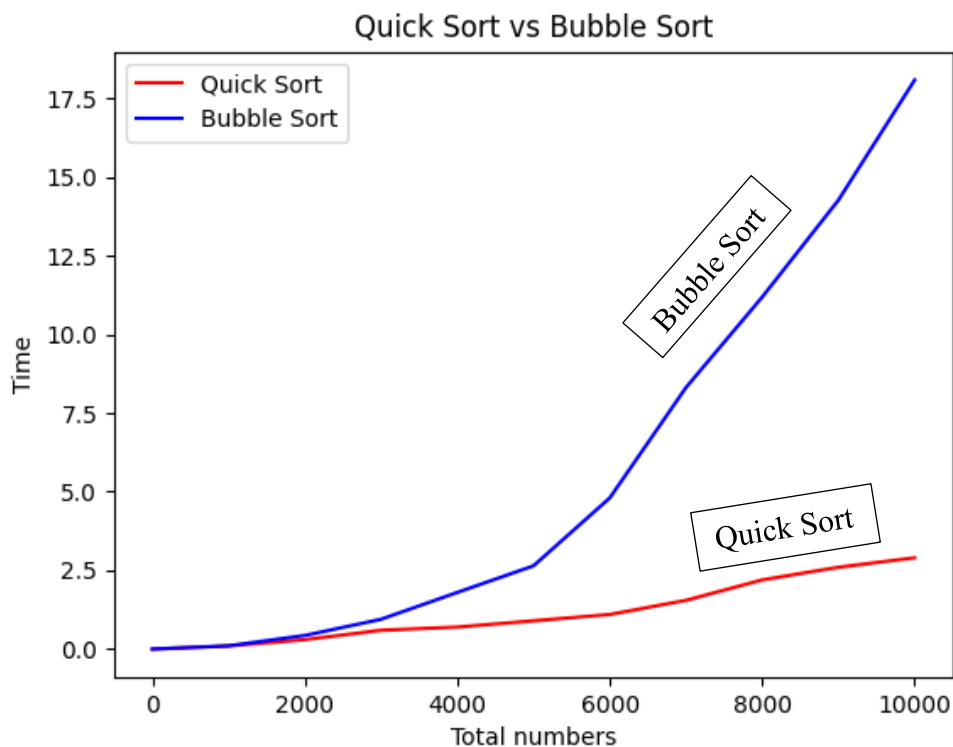
finish_time=datetime.now()
k=str(finish_time-start_time)
file.write(k+'\n')
B.clear()

```

Output:

FOR Quicksort	FOR BUBBLE SORT
0:00:00.000999	0:00:00.105000
0:00:00.002999	0:00:00.434235
0:00:00.006000	0:00:00.948037
0:00:00.007000	0:00:01.800259
0:00:00.009001	0:00:02.646585
0:00:00.009994	0:00:04.102048
0:00:00.011000	0:00:08.910971
0:00:00.021999	0:00:11.186535
0:00:00.024010	0:00:14.267181
0:00:00.027020	0:00:18.083052

Output.txt File



Discussion:

This is a very interesting task as there I used a file for getting inputs and after receiving outputs stored them in another file. For lessening the code lines, I have used OOP for developing the sorting algorithm. Then make an object of it and get access to the function of the class. Here I have used 10 inputs from 1000 to 10000 and stored them in descending order so that it can go to the worst case when ascending the numbers. Besides, I have broadened the scale 100 times for quick sorting to increase the visibility of that graph. After using the Python package 'matplotlib', I have drawn the graph for the comparison of the 2 sorting algorithms.